

FPGA - BASED INSULIN DOSAGE COMPUTATION SYSTEM FOR DIABETES MANAGEMENT

A Mini Project Report (Mini Project 2)

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN ELECTRONICS & COMMUNICATIONS ENGINEERING

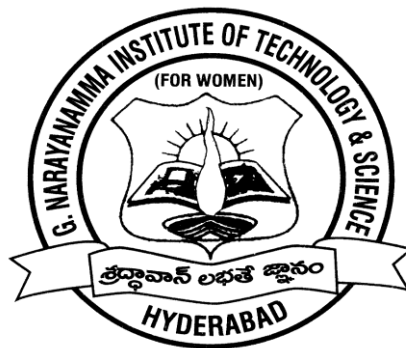
Submitted By

K. NAGA TEJA SRI	23251A04A7
M. TWINKLE DURGA	23251A04A8
S. VAISHNAVI	23251A04C2

Under the esteemed guidance of

Ms. P. Roopa Ranjani

Assistant Professor, ECE Dept, GNITS



**DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGG
G. NARAYANAMMA INSTITUTE OF TECHNOLOGY & SCIENCE**

(for women)

(AUTONOMOUS)

ACCREDITED by NBA & NAAC

2025 – 26

**DEPARTMENT OF ELECTRONICS & COMMUNICATION
ENGINEERING**

CERTIFICATE

THIS IS TO CERTIFY THAT THE MINI PROJECT ENTITLED

**FPGA - BASED INSULIN DOSAGE COMPUTATION SYSTEM FOR
DIABETES MANAGEMENT**

IS THE BONAFIDE WORK OF

K. NAGA TEJA SRI	23251A04A7
M. TWINKLE DURGA	23251A04A8
S. VAISHNAVI	23251A04C2

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE
AWARD OF DEGREE OF BACHELOR OF TECHNOLOGY IN ELECTRONICS AND
COMMUNICATION ENGINEERING DURING THE ACADEMIC YEAR 2025 - 26.

Ms. P. Roopa Ranjani
Asst. Prof.

Dr. K. Ragini
HOD - ECE

External Examiner

ACKNOWLEDGEMENTS

It is our privilege to express the feelings of gratitude to several people who helped us in completing the project.

We take this opportunity to express our heartfelt gratitude to our esteemed guide, **Ms. P. Roopa Ranjani**, Assistant Professor ECE department, GNITS for her constant guidance, encouragement, and immense motivation.

We are thankful to our Mini Project coordinator, **Ms. M. Madhuri Latha**, Assistant Professor, ECE department, GNITS for her valuable advice and encouragement.

We express our heartfelt gratitude to **Dr. K. Ragini**, Head of the Department, ECE department, GNITS for her constant guidance and moral support.

We would like to express our sincere thanks to **Dr. K. Ramesh Reddy**, Principal, GNITS for providing working facilities in the college.

Finally, we would like to thank all the faculty and staff of ECE department who helped us during our project and also our parents and friends for their cooperation in completing the project.

K. NAGA TEJA SRI	23251A04A7
M. TWINKLE DURGA	23251A04A8
S. VAISHNAVI	23251A04C2

INDEX

S. No.	Topic	Page No.
	Abstract	i
	List of figures	ii
	Introduction	
1.	1.1 Motivation	1
	1.2 Literature Survey	1
	1.3 Existing Methods	2
	1.4 Limitations of Existing Methods	3
	1.5 Problem Statement	4
	1.6 Proposed Method	4
	1.7 Objectives	5
	1.8 Methodology	6
2.	Hardware & Software Components	8
	2.1 Hardware Components	8
	2.1.1 FPGA Board	8
	2.1.2 Seven Segment Display	9
	2.1.3 USB Interface	10
	2.2 Software Tools	11
	2.2.1 Xilinx Vivado Tool	11
	2.2.2 Verilog HDL	12
3.	Implementation	13
4.	Results & Discussions	15
5.	Applications	21
6.	Conclusion & Future Scope	22
	References	23
	Appendix	24

ABSTRACT

Diabetes management requires accurate insulin delivery based on variations in blood glucose levels. Traditional insulin dosage methods often rely on manual calculation or basic software assistance, which may lead to delays, human errors. This project focuses on designing and implementing an automated insulin control prototype using an FPGA-based hardware system. In the proposed system, the insulin control logic is designed and implemented using Verilog HDL in Vivado and deployed on an FPGA platform.

A Finite State Machine (FSM) along with arithmetic logic is used to analyze glucose input values and generate the corresponding insulin output in a structured and deterministic manner. The system operates based on predefined parameters such as patient weight, age, previous insulin level, previous glucose level, and current glucose value, which help in estimating the required insulin dosage. The hardware implementation processes these parameters and produces an insulin output signal that represents the calculated insulin requirement.

Simulation & verification are performed in Vivado to ensure correct functionality of the control logic. The results demonstrate that the proposed FPGA-based framework can efficiently compute insulin dosage using parameter-based inputs and structured control logic. Although the system is not intended for direct clinical use, it serves as a proof-of-concept hardware model for automated insulin calculation and can be further extended in the future by integrating real glucose sensors & medical hardware interfaces for real-time diabetes management.

LIST OF FIGURES

S. No.	Fig. No.	Name of the Fig.	Page No.
1.	1.1	Block Diagram	7
2.	2.1	FPGA board Artix-7	8
3.	2.2	Seven Segment Display	9
4.	2.3	USB Interface	10
5.	2.4	Xilinx Vivado	11
6.	4.1	I/P Parameters for Insulin Computation	15
7.	4.2	PLL Output Waveform	16
8.	4.3	FSM Control Output	16
9.	4.4	RAM Output Waveform	17
10.	4.5	Insulin Computation Module Waveform	17
11.	4.6	Top Module Output Waveform	18
12.	4.7	Schematic Diagram	18
13.	4.8	FPGA Constraint XDC File	19
14.	4.9	FPGA Hardware Implementation	19

1. INTRODUCTION

Diabetes is a long-term condition that requires the continuous monitoring of glucose levels in the blood. This requires the accurate administration of insulin to maintain normal body function. The incorrect administration of insulin may lead to critical health conditions. Therefore, the accurate calculation of insulin is of critical importance to manage diabetes correctly.

In the proposed project, an automated insulin control system is proposed with the help of FPGA. In this project, Verilog HDL and FSM logic have been used to calculate the amount of insulin required in the body depending on the parameters like the present glucose levels, previous glucose levels, weight, age, and previous levels of insulin. Therefore, the proposed project demonstrates the parameters that need to be considered for the efficient calculation of insulin levels in the body.

1.1 MOTIVATION

Diabetes patients require insulin to be administered according to their blood glucose levels, which are ever-changing. Conventional solutions involve manually performing calculations or employing simple computer programs to achieve this, which can result in delay and human error, potentially causing incorrect dosage.

The motivation behind this project is to develop a solution that can meet the requirement of a fast, efficient, and automated solution to insulin dosage, which can be achieved through a hardware solution involving an FPGA with a control logic based on FSM. This can effectively handle multiple patient parameters in a structured manner.

1.2 LITERATURE SURVEY

Recent advancements in diabetes management have focused on automated insulin delivery systems using both software and hardware approaches. Stanchieri et al. proposed an FPGA-based digital insulin-glucose regulator that performs insulin computation using a fully hardware-based architecture, eliminating the need for software and enabling fast, deterministic operation [1]. The system was validated using virtual patient models and achieved high accuracy

with error less than $\pm 1\%$ and low power consumption, making it suitable for real-time biomedical applications.

In addition to FPGA-based solutions, several studies have explored closed-loop insulin delivery systems, commonly known as artificial pancreas systems. Hovorka [2] and Doyle et al. [3] discussed the development of closed-loop control algorithms that automatically regulate insulin based on continuous glucose monitoring. These systems improve patient outcomes but involve complex algorithms and expensive medical devices.

Further research has also investigated FPGA and embedded system implementations for biomedical applications. Pandey and Nagaria [4] designed an FPGA-based insulin pump controller demonstrating efficient hardware computation, while Alioto [5] highlighted the advantages of FPGA technology such as parallel processing, low power consumption, and high reliability in medical systems. Compared to these works, the proposed project focuses on a simplified FPGA-based insulin control system using FSM and parameter-based inputs, providing a cost-effective and scalable solution suitable for academic and future real-time applications

1.3 EXISTING METHOD

The existing methods for calculating the insulin dosage include manual methods, mobile technology, and advanced technology using insulin pumps and artificial pancreas. Although the advanced technology provides the advantage of automated calculation of the insulin dosage, it is considered an expensive and complex technology that requires sophisticated sensors for the proper functionality of the system.

The proposed system is based on a simple method using FPGA technology with the help of Verilog HDL. The benefits of the proposed system include faster computation, parallel processing, and reliability with the use of predefined parameter.

1.4 LIMITATIONS OF EXISTING SYSTEM

Table 1.1- Limitations and its Description

Limitations	Description
High cost	This system incorporates advanced technologies such as AI, GPS, and robotics. This makes it inaccessible to small-scale farmers.
Technical complexity	This system requires skilled manpower for handling and troubleshooting.
GPS-Dependency	This system may not be effective in areas where there is no or poor availability of GPS signal, for instance, indoor farms or hilly terrain.
Limited accessibility	This system is not accessible or appropriate for farmers in rural or developing countries.
Overengineered for Small Fields	This system is too complex or too large for small or segmented farmland.

1.5 PROBLEM STATEMENT

Accurate computation of the insulin dosage is of utmost importance to successfully control diabetes, as incorrect dosages may cause severe health issues. The current approach relies on manual calculations and/or the use of software tools, which may cause errors due to the involvement of the human factor, delays, and lack of response to queries during runtime. Although advanced automated systems are available and are quite effective, they are complex, expensive, and cannot be implemented for low-cost and academic purposes.

In this context, the need to develop an accurate, fast, and cost-effective system to automatically compute the insulin dosage based on several patient characteristics becomes evident. The proposed project will develop an insulin control system based on the FPGA platform, implemented by Verilog HDL and FSM logic, to provide a structured, deterministic, and efficient solution to automatically compute the insulin dosage.

1.6 PROPOSED METHOD

The proposed system is an automated insulin control system using an FPGA that calculates the insulin dose using hardware logic. The system is implemented using the Verilog HDL, which is simulated using the Vivado environment, along with a Finite State Machine that processes the input parameters, i.e., the current glucose level, previous glucose level, patient weight, patient age, and previous insulin levels, etc. The input parameters are then processed by the system, performing arithmetic operations, and an appropriate insulin output signal is generated.

Unlike the traditional software implementation, the proposed system is much faster, as the operations are performed in parallel, making the system highly efficient and reliable. The system is developed as a proof of concept model, which is simulated using the Vivado environment, and the system is verified accordingly. The system may be extended by adding the real-time glucose sensors, etc., for the actual implementation of the system in the field of diabetes management.

1.7 OBJECTIVES

The purpose of this project is to develop an FPGA-based insulin control system that uses the input glucose and other parameters of the patient and generates accurate insulin dosages using hardware structures.

To Process Glucose and Patient Inputs

- Accept the input glucose value, previous value of glucose, and other parameters of the patient.
- Proper handling and continuous monitoring of input values are essential.

To Implement Clock Generation using PLL

- Clock generation is essential for the proper functioning of the system.
- Synchronous clock generation is done using the PLL technique. Reliable clock generation is essential.

To Design FSM-Based Control Unit

- The FSM is designed to implement the control unit. FSM is essential in the implementation of the system.
- It is designed in such a way that the system is executed in a structured manner.

To Perform Insulin Dosage Calculation

- The calculation of the amount of insulin is done using arithmetic operations.
- The accuracy of the result depends on the input parameters.

To Implement Data Storage and FPGA Implementation

- The previous data is stored using Dual Port RAM. This is done in an efficient manner.
- The complete system is implemented on FPGA using Verilog HDL. Simulation is done

using Vivado tools.

1.8 METHODOLOGY

The proposed system follows a structured hardware-based approach for automated insulin calculation using an FPGA platform. Initially, input parameters such as current glucose level, previous glucose level, and patient-specific data are provided to the system. These inputs are processed inside the FPGA module, which consists of multiple functional units designed to perform synchronized and efficient computation.

The FPGA module begins operation with the PLL (Phase Locked Loop), which generates the required clock signals for proper timing and synchronization of all internal operations. The FSM (Finite State Machine) acts as the control unit, managing the sequence of operations such as input processing, data transfer, and insulin calculation. The Insulin Calculation Module performs arithmetic computations based on the input parameters to determine the required insulin dosage. Simultaneously, the Dual Port RAM is used to store previous glucose values and insulin data, enabling efficient read and write operations for continuous system functioning.

Finally, after processing all inputs and calculations, the system produces the insulin dosage output, which represents the required insulin level for the patient. The entire process is implemented using Verilog HDL and verified using Vivado tools, ensuring accurate, fast, and reliable hardware-based computation.

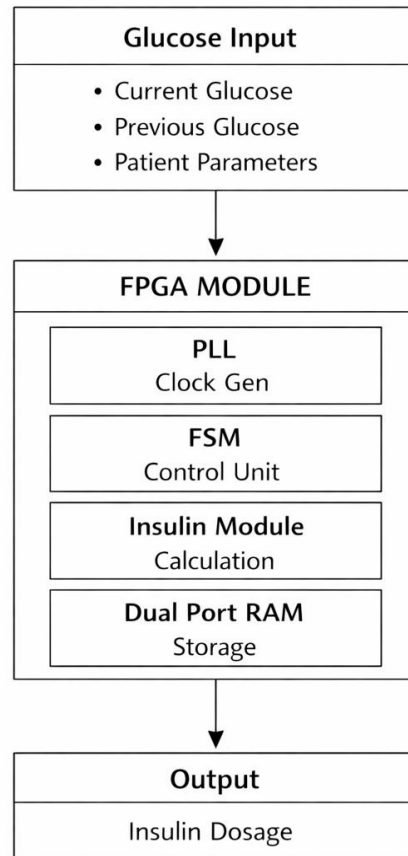


Fig. 1.1 BLOCK DIAGRAM

The block diagram represents the overall architecture of the FPGA-based insulin control system. It shows the flow of input parameters such as glucose levels and patient data into the FPGA module, where different units like PLL, FSM, insulin calculation module, and memory perform processing. The final output is the calculated insulin dosage.

2. HARDWARE & SOFTWARE COMPONENTS

2.1 HARDWARE COMPONENTS

- Xilinx Artix-7 FPGA
- Seven Segment Display
- USB Interface

2.1.1 FPGA Board

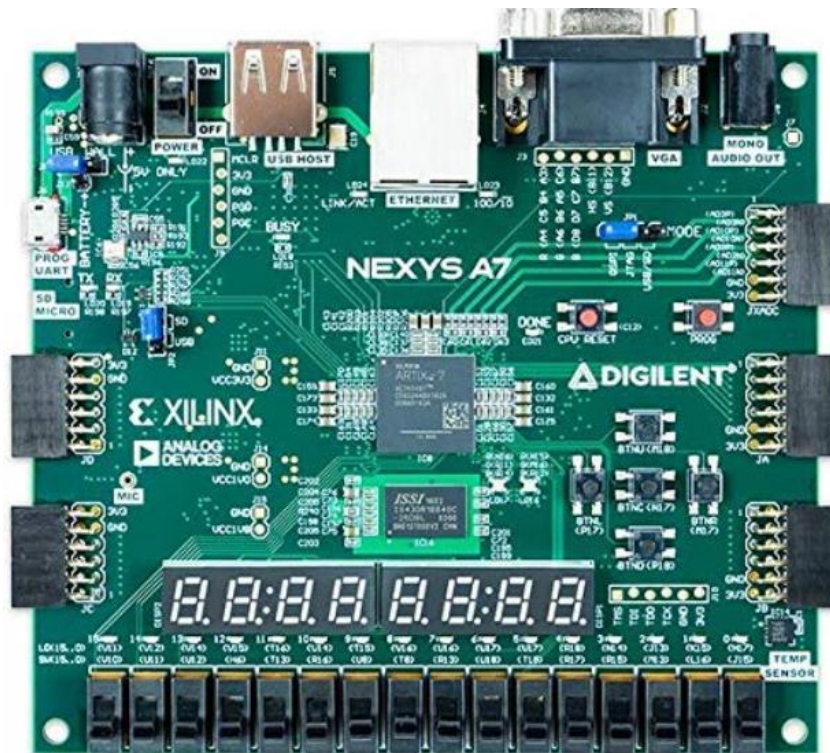


Fig. 2.1 FPGA Board – ARTIX 7

The proposed system is implemented using the **Nexys A7 FPGA development board**, which is based on the Xilinx Artix-7 FPGA. This board serves as the core hardware platform for executing the insulin control logic designed using Verilog HDL. The Artix-7 FPGA provides high-speed processing, parallel execution capability, and efficient resource utilization, making it suitable for real-time biomedical applications.

The board is powered and programmed through a **USB interface**, which connects the FPGA to a computer running Vivado software. This interface is used to upload the hardware design and also supplies power to the system. The board includes an **onboard clock oscillator**, which generates the clock signal required for synchronization of all internal operations. This clock is further managed using a PLL inside the FPGA to ensure stable and accurate timing.

For user interaction, the board provides **slide switches and push buttons**, which are used to input parameters such as glucose levels and control signals like reset or start. The **7-segment display** and **LEDs** are used to display output values such as insulin dosage and system status, making it easier to observe system behavior in real time.

Additionally, the FPGA contains **internal Block RAM (Dual Port RAM)**, which is used to store previous glucose values and insulin data for continuous processing. Other peripherals like **VGA and Ethernet ports** are available on the board but are not utilized in this project.

2.1.2 Seven Segment Display



Fig. 2.2 Seven Segment Display

The Nexys A7 FPGA board is equipped with multiple **7-segment displays**, which are used to display numerical output values. Each 7-segment display consists of seven individual LEDs arranged in the shape of the number “8,” along with an additional decimal point. By turning ON or OFF specific segments, different digits (0–9) can be displayed.

In this project, the 7-segment display is used to show the **calculated insulin dosage** generated by the FPGA system. The output from the insulin calculation module is sent to a display control unit, which converts binary values into corresponding segment signals. Since the FPGA produces digital outputs, encoding logic is used to map numerical values to the correct combination of segments.

The display operates using a technique called **multiplexing**, where multiple digits are displayed rapidly one after another, giving the illusion of continuous output. This reduces hardware usage while allowing multiple values to be displayed simultaneously.

Thus, the 7-segment display provides a simple and effective way to visualize system outputs in real time, making it useful for monitoring and debugging the insulin control system.

2.1.3 USB Interface



Fig. 2.3 USB Interface

The Nexys A7 FPGA board includes a built-in **USB interface**, which serves two main purposes: **programming the FPGA** and **providing power to the system**. The board is connected to a computer using a USB cable, through which the design created in Vivado (in the form of a bitstream file) is downloaded into the FPGA. This process configures the FPGA to implement the insulin control logic designed using Verilog HDL.

In addition to programming, the USB interface also supplies the required power to operate the FPGA board, eliminating the need for an external power source. The interface typically uses a **JTAG (Joint Test Action Group) protocol**, which enables communication between the computer and FPGA for configuration and debugging.

2.2 SOFTWARE TOOLS

- Xilinx Vivado
- Verilog HDL (Language)

2.2.1 Xilinx Vivado



Fig. 2.4 Vivado Tool

Vivado Design Suite is the primary software tool used for designing and implementing FPGA-based systems in this project. It is developed by Xilinx and provides a complete environment for writing, simulating, synthesizing, and deploying hardware designs. In this project, Vivado is used to create and manage the Verilog HDL code for the insulin control system.

The design process in Vivado includes multiple stages. First, the Verilog code is written and verified using the **Vivado Simulator**, which checks the correctness of the logic and output for different input conditions. After simulation, the design is passed through **synthesis**, where the HDL code is converted into a hardware-level representation. This is followed by **implementation**, where the design is mapped onto FPGA resources such as LUTs, flip-flops, and memory blocks.

Finally, Vivado generates a **bitstream file**, which is used to program the FPGA board through the USB interface. This allows the designed insulin control system to run on actual hardware. Vivado also provides debugging and analysis tools, making it essential for efficient FPGA development.

2.2.2 Verilog HDL

Verilog HDL (Hardware Description Language) is used in this project to design and implement the digital logic of the FPGA-based insulin control system. It allows the system to be described at the hardware level using modules, enabling the design of components such as the Finite State Machine (FSM), insulin calculation unit, and memory blocks.

Verilog supports both **behavioral and structural modeling**, making it suitable for designing complex digital systems. In this project, Verilog is used to define the flow of operations, control logic, and arithmetic computations required for insulin dosage calculation. The modular nature of Verilog helps in organizing the design into smaller functional units, improving readability and reusability.

Additionally, Verilog enables **parallel execution**, which is a key advantage in FPGA-based systems, allowing faster processing compared to sequential software programs. The written code is simulated and verified in Vivado before being synthesized into hardware. Thus, Verilog HDL plays a crucial role in converting the system design into a functional hardware implementation.

3. IMPLEMENTATION

Software Implementation

The software implementation involves designing, coding, and verifying the insulin control system using Vivado and Verilog HDL.

- **Design Entry**

The system architecture is designed based on the block diagram, including modules such as FSM, insulin calculation unit, PLL, and memory.

- **Coding in Verilog HDL**

The complete logic of the system is written using Verilog HDL. Individual modules are created for FSM control, arithmetic operations, and data storage.

- **Simulation**

The design is tested using the Vivado Simulator. Different input conditions (glucose values, parameters) are applied to verify correct insulin output.

- **Synthesis**

The verified design is converted into a hardware-level netlist using Vivado synthesis tools.

- **Implementation**

The synthesized design is mapped onto FPGA resources such as LUTs, flip-flops, and RAM.

- **Bitstream Generation**

A bitstream (.bit file) is generated, which contains the final configuration data for the FPGA.

Hardware Implementation

The hardware implementation involves deploying the design onto the FPGA board and testing it in real-time.

- **FPGA Setup**

The Nexys A7 FPGA board is connected to the computer using a USB cable for power and programming.

- **Programming the FPGA**

The generated bitstream file is uploaded to the FPGA using Vivado.

- **Input Configuration**

Input parameters such as glucose levels are provided through switches or predefined test inputs.

- **System Execution**

The FPGA executes the Verilog design, where FSM controls the process, and the insulin calculation module computes the output.

- **Data Storage**

Dual Port RAM stores previous glucose and insulin values for continuous processing.

- **Output Display**

The calculated insulin dosage is displayed using LEDs or 7-segment display.

- **Testing and Verification**

The hardware output is observed and verified to ensure correct system performance.

4. RESULTS & DISCUSSIONS

The simulation results verify the proper functionality of the proposed FPGA-based insulin dosage computation system, which has five modules: PLL, Insulin Module, FSM, RAM, and control logic. The system has five input parameters, namely, age, weight, current glucose level, previous glucose level, and previous insulin dosage, for computation.

The insulin module computes the insulin dosage, and the FSM manages the execution of the system. The PLL generates the clock for the system. The computed insulin dosage is then stored in the RAM and passed on through the control logic. The waveform results show the accurate computation of insulin dosage, thus verifying the proper integration of all modules in the system.

```
weight          = 16'd70;
age             = 16'd30;
prev_insulin    = 16'd10;
prev_glucose    = 16'd7;
current_glucose = 16'd10;
```

Fig. 4.1 Input Parameters for Insulin Computation

Fig. 4.1 shows the input parameters provided to the FPGA-based system, including weight, age, previous insulin dosage, previous glucose level, and current glucose level. These values are defined in the testbench as fixed inputs and are applied to the insulin module during simulation. Based on these parameters, the system processes the data to compute the required insulin dosage. This setup allows verification of the design under specific conditions and ensures correct functionality of the computation logic.

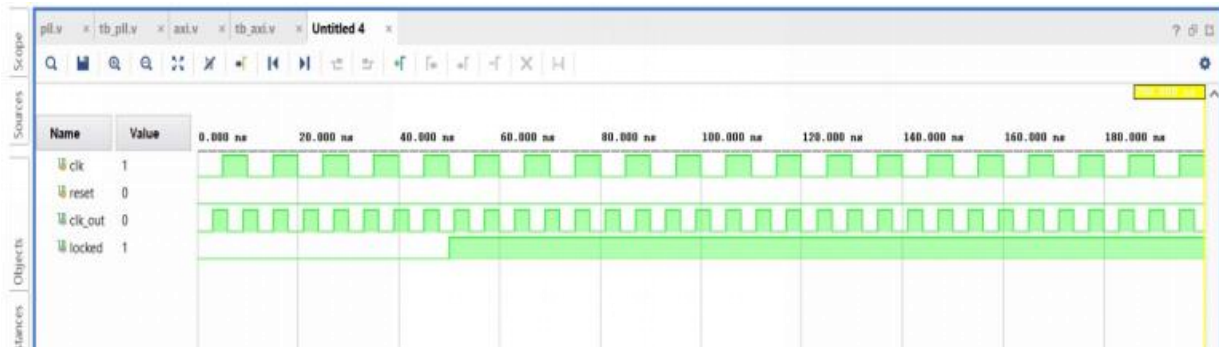


Fig. 4.2 PLL Output Waveform

Fig. 4.2 shows the simulation waveform of the PLL module generating a stable clock signal from the input clock. The output clock (clk_out) is synchronized and the locked signal confirms stable operation of the system. This ensures reliable timing for all modules in the FPGA design.

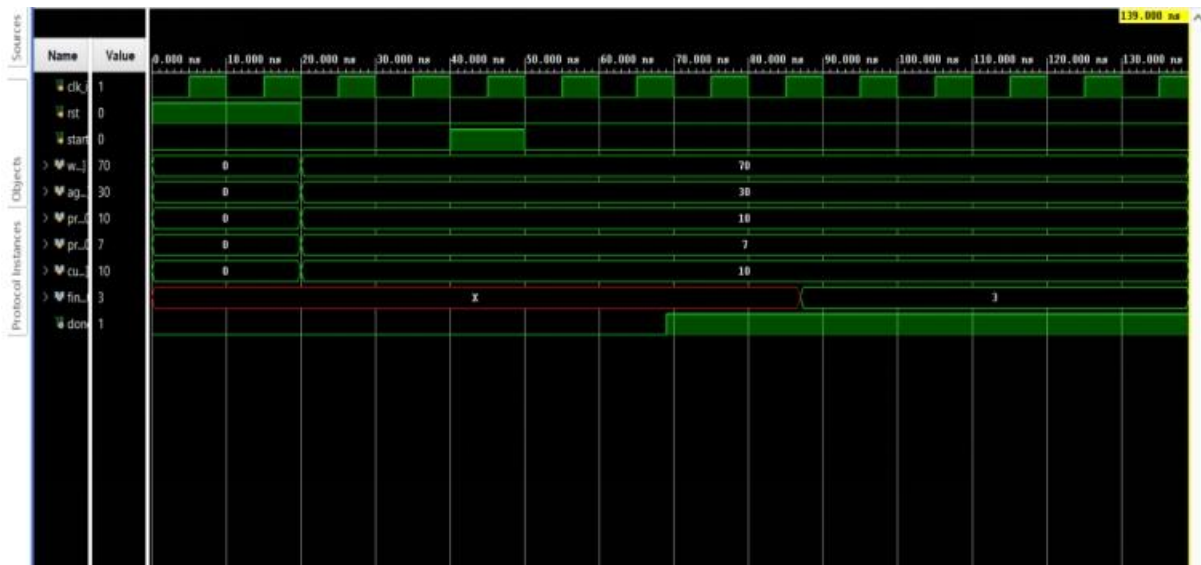


Fig. 4.3 FSM Control Output Waveform

Fig. 4.3 shows the FSM-controlled operation of the insulin dosage computation system. The input parameters such as weight, age, prev_insulin, prev_glucose, and current_glucose are assigned in the testbench and processed by the insulin module. Control signals like clk, rst, and start drive the FSM, which manages the execution flow of the system. The final_insulin output is generated based on these inputs and stabilizes after computation, while the done signal indicates successful completion of the process.

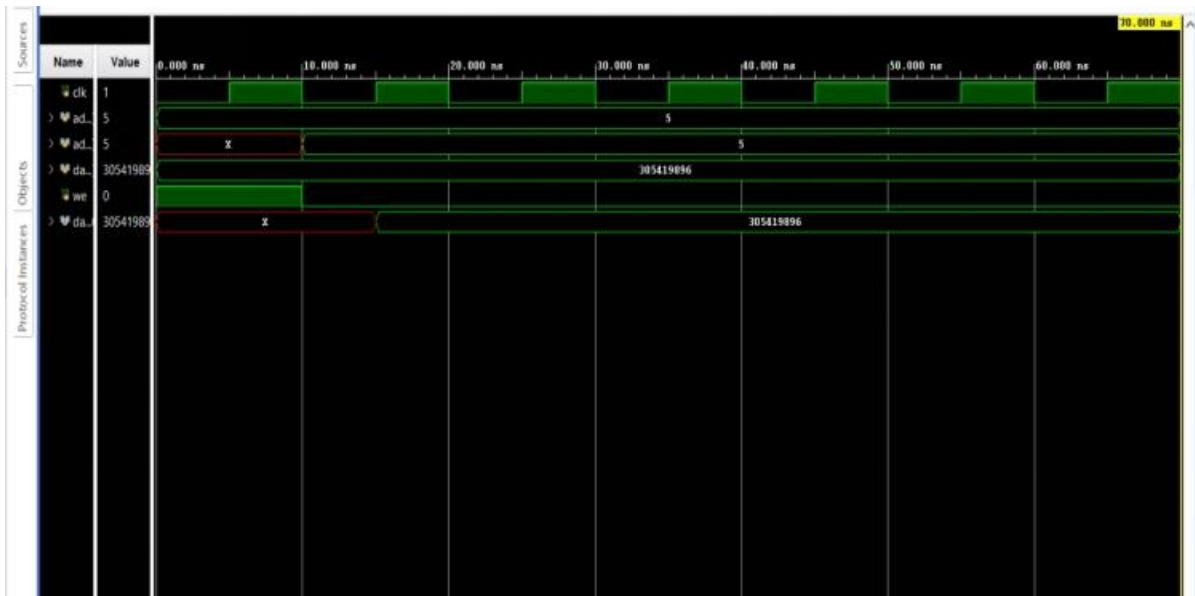


Fig. 4.4 RAM Output Waveform

Fig. 4.4 shows the operation of the dual-port RAM module. The waveform illustrates read and write activities using signals such as data_in and data_out. The stored data is successfully retrieved after initial undefined states, confirming correct memory operation. The RAM module acts as temporary storage for the computed insulin values, enabling efficient data transfer between the insulin module and other components in the system.

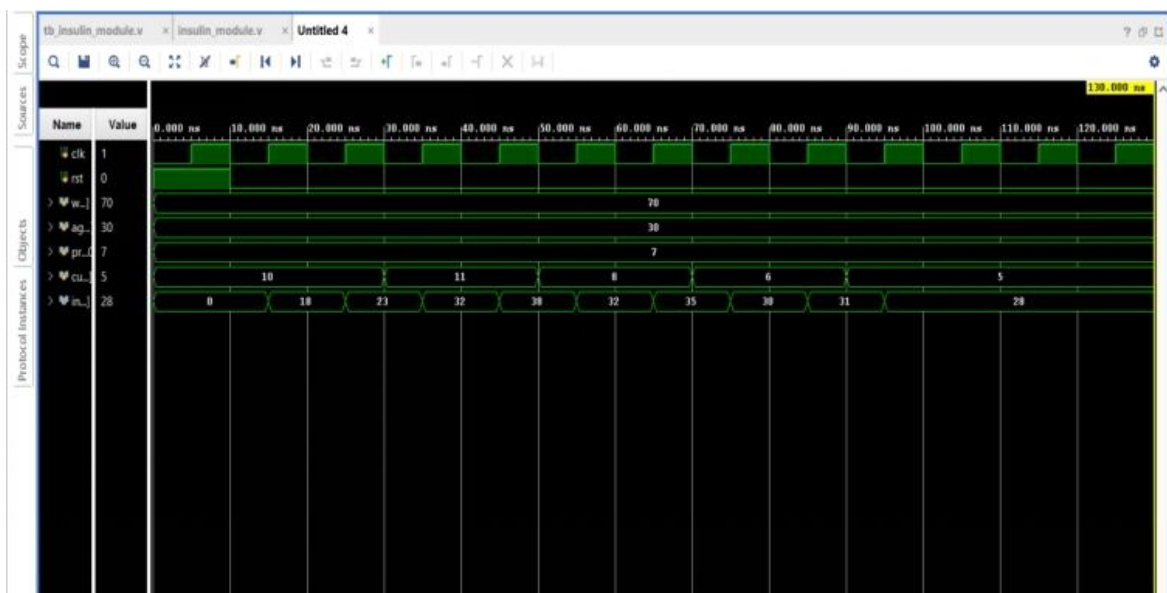


Fig. 4.5 Insulin Computation Module Waveform
GNITS - ECE DEPT

Fig. 4.5 shows the simulation waveform of the insulin module for given input parameters such as age, weight, and glucose levels. These inputs are provided through the testbench and processed by the module to compute the required insulin dosage. The waveform demonstrates how the insulin_out signal changes based on varying input conditions, confirming correct implementation of the computation logic in the design.



Fig. 4.6 Top Module Output Waveform

Fig. 4.6 shows the simulation waveform of the insulin module for given input parameters such as age, weight, and glucose levels. These inputs are provided through the testbench and processed by the module to compute the required insulin dosage. The waveform demonstrates how the insulin_out signal changes based on varying input conditions, confirming correct implementation of the computation logic in the design.

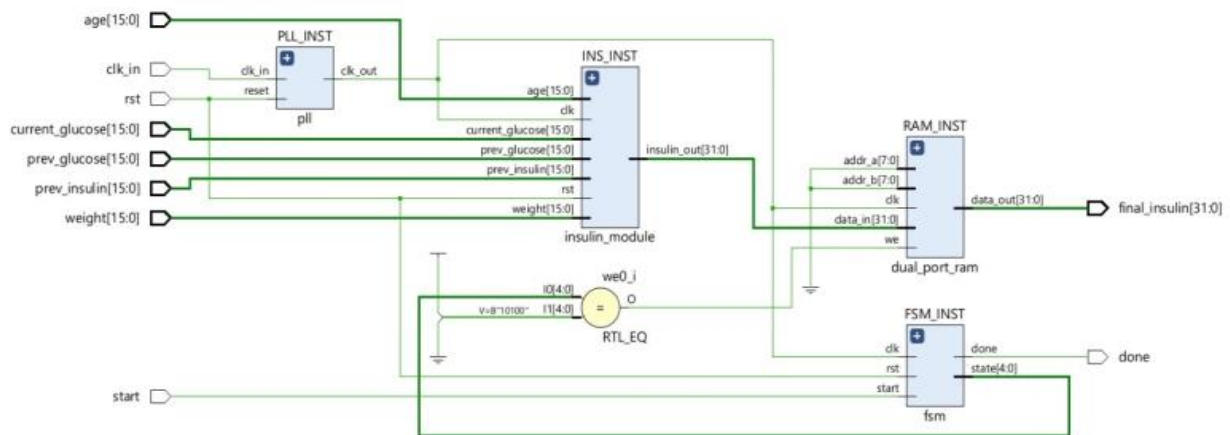


Fig. 4.7 Schematic Diagram
GNITS - ECE DEPT

Fig. 4.7 shows the overall architecture of the FPGA-based insulin dosage computation system. The design consists of five main modules: PLL, Insulin Module, FSM, RAM, and control logic. Input parameters such as age, weight, current glucose, previous glucose, and previous insulin are provided to the insulin module for processing.

```

1  set_property IOSTANDARD LVCMOS33 [get_ports {final_insulin[7]}]
2  set_property IOSTANDARD LVCMOS33 [get_ports {final_insulin[6]}]
3  set_property IOSTANDARD LVCMOS33 [get_ports {final_insulin[5]}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {final_insulin[4]}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {final_insulin[3]}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {final_insulin[2]}]
7  set_property IOSTANDARD LVCMOS33 [get_ports {final_insulin[1]}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {final_insulin[0]}]
9  set_property IOSTANDARD LVCMOS33 [get_ports rst]
10 set_property IOSTANDARD LVCMOS33 [get_ports done]
11 set_property IOSTANDARD LVCMOS33 [get_ports clk_in]
12 set_property PACKAGE_PIN E3 [get_ports clk_in]
13 set_property PACKAGE_PIN V10 [get_ports rst]
14 set_property PACKAGE_PIN U16 [get_ports {final_insulin[0]}]
15 set_property PACKAGE_PIN V16 [get_ports {final_insulin[1]}]
16 set_property PACKAGE_PIN T15 [get_ports {final_insulin[2]}]
17 set_property PACKAGE_PIN U14 [get_ports {final_insulin[3]}]
18 set_property PACKAGE_PIN T16 [get_ports {final_insulin[4]}]
19 set_property PACKAGE_PIN V15 [get_ports {final_insulin[5]}]
20 set_property PACKAGE_PIN V14 [get_ports {final_insulin[6]}]
21 set_property PACKAGE_PIN V12 [get_ports {final_insulin[7]}]
22 set_property PACKAGE_PIN V11 [get_ports done]
23

```

Fig. 4.8 FPGA Constraint File (XDC)

Fig. 4.8 shows the XDC constraint file used for mapping input and output ports to FPGA pins. It defines I/O standards and pin assignments for signals such as final_insulin, clk_in, rst, and done, enabling proper hardware implementation on the Nexys A7 board.

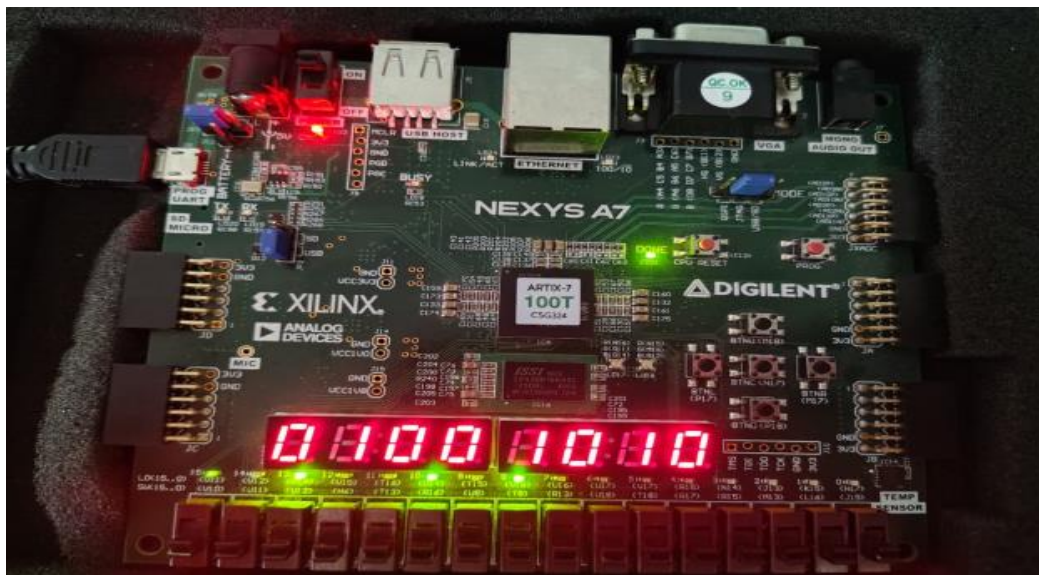


Fig. 4.9 FPGA Hardware Implementation
GNITS - ECE DEPT

Fig. 4.9 shows the Nexys A7 FPGA board used for implementing the insulin dosage computation system. The computed insulin value is displayed using the on-board seven-segment display and LEDs. Since the FPGA design operates using integer representation, the displayed value “63” corresponds to 6.3 IU of insulin. This confirms the successful hardware realization and output visualization of the system.

5. APPLICATIONS

- Used in diabetes management systems to calculate accurate insulin dosage.
- Assists patients and healthcare professionals in making better treatment decisions.
- Reduces human errors by automating insulin calculation.
- Provides fast and reliable results through hardware-based processing.
- Applicable in medical decision support systems requiring real-time data processing.
- Can be integrated with glucose monitoring devices for continuous tracking.
- Supports IoT-based healthcare systems for remote patient monitoring.
- Enables timely alerts and automated insulin recommendations.
- Suitable for embedded and wearable healthcare devices due to FPGA efficiency.

6. CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The FPGA-based insulin dosage computation system was successfully designed and implemented using Verilog HDL. The system processes key input parameters such as age, weight, and glucose levels to generate an appropriate insulin output. Simulation results verified the correct functionality of individual modules as well as the complete integrated system.

The hardware implementation on the Nexys A7 FPGA board further confirms the practical feasibility of the design. The computed output displayed on the seven-segment display validates the real-time operation of the system. The use of integer representation for insulin values ensures compatibility with hardware constraints.

Overall, the project demonstrates the effectiveness of FPGA-based solutions for fast, reliable, and accurate medical computations. It provides a strong foundation for future enhancements such as integration with real-time sensors and advanced healthcare systems.

6.2 Future Scope

The proposed system can be further enhanced by integrating real-time glucose sensing devices, enabling continuous monitoring and automatic insulin dosage computation without the need for manual input. This would make the system more practical and applicable in real-world healthcare scenarios. Additionally, support for higher precision or floating-point computation can be incorporated to improve the accuracy of insulin dosage representation beyond integer-based values.

Furthermore, the system can be extended by integrating IoT and mobile-based applications for remote monitoring, data logging, and communication with healthcare professionals. It can also be developed into a complete smart insulin delivery system by interfacing with insulin pumps and wearable devices. Incorporating advanced algorithms and machine learning techniques can enable personalized and adaptive insulin recommendations, improving overall patient care and treatment efficiency.

REFERENCES

- [1] G. Di Patrizio Stanchieri et al., “FPGA-Based Implementation of a Digital Insulin-Glucose Regulator for Type 2 Diabetic Patients,” *Electronics*, vol. 13, no. 9, 2024.
- [2] R. Hovorka, “Closed-loop insulin delivery: From concept to clinical practice,” *Nat. Rev. Endocrinol.*, vol. 7, no. 7, pp. 385–395, 2011.
- [3] F. J. Doyle III et al., “Closed-loop artificial pancreas systems: Engineering the algorithms,” *Diabetes Care*, vol. 37, no. 5, pp. 1191–1197, 2014.
- [4] S. K. Pandey and R. K. Nagaria, “FPGA-based design of insulin pump controller,” *Int. J. Eng. Res. Technol. (IJERT)*, vol. 3, no. 6, pp. 1–5, 2014.
- [5] M. Alioto, “Design of digital systems for biomedical applications using FPGA,” *IEEE Trans. Biomed. Circuits Syst.*, vol. 10, no. 3, pp. 1–10, 2016.
- [6] E. Dassau et al., “Artificial pancreas systems: Current status and future prospects in the management of diabetes,” *IEEE Rev. Biomed. Eng.*, vol. 5, pp. 53–72, 2012.
- [7] C. Cobelli, C. Dalla Man, G. Sparacino, L. Magni, G. De Nicolao, and B. P. Kovatchev, “Diabetes: Models, signals, and control,” *IEEE Rev. Biomed. Eng.*, vol. 2, pp. 54–96, 2009.
- [8] S. Majumder, T. Mondal, and M. J. Deen, “Wearable sensors for remote health monitoring,” *IEEE Sensors J.*, vol. 17, no. 5, pp. 130–148, 2017.
- [9] P. Kumaresan and M. S. Bhat, “FPGA implementation of biomedical signal processing systems,” *Int. J. Adv. Res. Comput. Eng. Technol.*, vol. 4, no. 2, pp. 1–6, 2015.

APPENDIX

PLL(Phase Locked Loop)

```
module pll(  
    input wire clk_in,  
    input wire reset,  
    output reg clk_out,  
    output reg locked  
);  
initial begin  
    clk_out = 0;  
    locked = 0;  
    #50 locked = 1;  
end  
always #3 clk_out = ~clk_out; // ~166 MHz  
endmodule
```

FSM(Finite State Machine)

```
module fsm(  
    input clk,  
    input rst,  
    input start,  
    output reg done,  
    output reg [4:0] state  
);  
parameter IDLE = 5'd0;  
parameter LOAD = 5'd1;  
parameter CALC1 = 5'd2;  
parameter CALC2 = 5'd3;
```

```

parameter WRITE = 5'd20;
reg [4:0] next_state;
always @(posedge clk or posedge rst) begin
    if (rst)
        state <= IDLE;
    else
        state <= next_state;
end
always @(*) begin
    next_state = state;
    case(state)
        IDLE:
            if(start)
                next_state = LOAD;
        LOAD:
            next_state = CALC1;
        CALC1:
            next_state = CALC2;
        CALC2:
            next_state = WRITE;
        WRITE:
            next_state = IDLE;
        default:
            next_state = IDLE;
    endcase
end
always @(posedge clk or posedge rst) begin
    if(rst)
        done <= 0;
    else

```

```

        done <= (state == WRITE);
end
endmodule

```

INSULIN MODULE

```

module insulin_module (
    input wire    clk,
    input wire    rst,
    input wire [15:0] weight,
    input wire [15:0] age,
    input wire [15:0] prev_insulin,
    input wire [15:0] prev_glucose,
    input wire [15:0] current_glucose,
    output reg [31:0] insulin_out
);
parameter TARGET = 5;
parameter KP = 3;
parameter KI = 1;
reg signed [31:0] error;
reg signed [31:0] integral;
reg signed [31:0] delta_glucose;
reg signed [31:0] insulin_calc;
always @(posedge clk or posedge rst)
begin
    if(rst)
    begin
        insulin_out <= 0;
        integral <= 0;
    end
    else

```

```

begin
    error <= current_glucose - TARGET;
    delta_glucose <= current_glucose - prev_glucose;
    integral <= error;
    insulin_calc <= prev_insulin +
        (KP * error) +
        (KI * integral) +
        delta_glucose;
    insulin_out <= insulin_calc;
end
end
endmodule

```

DUAL PORT RAM

```

module dual_port_ram(
    input clk,
    input [7:0] addr_a,
    input [7:0] addr_b,
    input [31:0] data_in,
    input we,
    output reg [31:0] data_out
);
    reg [31:0] memory [255:0];
    always @(posedge clk)
    begin
        if (we)
            memory[addr_a] <= data_in;

        data_out <= memory[addr_b];
    end
end

```

```
endmodule
```

TOP INSULIN SYSTEM

```
module top_insulin_system (  
    input wire    clk_in,  
    input wire    rst,  
    input wire    start,  
    input wire [15:0] weight,  
    input wire [15:0] age,  
    input wire [15:0] prev_insulin,  
    input wire [15:0] prev_glucose,  
    input wire [15:0] current_glucose,  
    output wire [31:0] final_insulin,  
    output wire    done  
);  
wire clk_fast;  
wire pll_locked;  
pll PLL_INST(  
    .clk_in(clk_in),  
    .reset(rst),  
    .clk_out(clk_fast),  
    .locked(pll_locked)  
);  
wire [4:0] state;  
fsm FSM_INST(  
    .clk(clk_fast),  
    .rst(rst),  
    .start(start),  
    .done(done),  
    .state(state)
```

```

);
wire [31:0] insulin_value;
insulin_module INS_INST(
    .clk(clk_fast),
    .rst(rst),
    .weight(weight),
    .age(age),
    .prev_insulin(prev_insulin),
    .prev_glucose(prev_glucose),
    .current_glucose(current_glucose),
    .insulin_out(insulin_value)
);
wire [31:0] ram_out;
dual_port_ram RAM_INST(
    .clk(clk_fast),
    .addr_a(8'd0),
    .addr_b(8'd0),
    .data_in(insulin_value),
    .we(state == 5'd20),
    .data_out(ram_out)
);
assign final_insulin = ram_out;
endmodule

```